

Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

Hands-On #16

IPC in Linux: semafori POSIX e memoria condivisa

Last Update: May 11, 2026

Obiettivi

In questo HO implementeremo un meccanismo fondamentali di **Inter-Process Communication** tra processi non imparentati:

- **Semafori POSIX con nome** — per sincronizzare due processi indipendenti attraverso `/dev/shm`, con gestione corretta del ciclo di vita e della pulizia in caso di crash;

Mappa concettuale

Ciclo di vita di una risorsa IPC con nome

Semafori POSIX con nome seguono questo schema:

```
sem_open / shm_open  →  uso  →  close + unlink
(crea o apre)                (rimuove il nome)
```

La `unlink` rimuove il nome da `/dev/shm`, ma la risorsa rimane in vita finché tutti i file descriptor che la referenziano vengono chiusi. **Senza `unlink` la risorsa sopravvive al reboot.**

Esercizi

Esercizio 1 — Semafori POSIX con nome tra processi

Esercizio 1 — Writer e reader sincronizzati

Scrivere **due programmi separati**, `writer.c` e `reader.c`, che comunicano tramite un file binario in `/tmp` e si sincronizzano con due semafori POSIX con nome:

- `/sem_ready` (valore iniziale 0): il reader lo incrementa per segnalare al writer che è pronto a ricevere;
- `/sem_done` (valore iniziale 0): il writer lo incrementa dopo aver scritto, per segnalare al reader che può leggere.

Flusso atteso:

1. il **reader** parte, crea entrambi i semafori con `O_CREAT`, fa `sem_post(/sem_ready)` e poi attende su `sem_wait(/sem_done)`;
2. il **writer** parte, attende `sem_wait(/sem_ready)`, scrive 100 interi casuali in `/tmp/dati.bin`, poi fa `sem_post(/sem_done)`;
3. il reader si sveglia, legge il file e stampa il valore massimo.

Entrambi i programmi devono installare un handler per `SIGINT` che esegue la pulizia (`sem_close`, `sem_unlink`, rimozione del file) prima di terminare.

Listing 1: writer.c – da completare

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <semaphore.h>
5  #include <signal.h>
6  #include <unistd.h>
7  #include <time.h>
8
9  #define SEM_READY  "/sem_ready"
10 #define SEM_DONE   "/sem_done"
11 #define DATAFILE  "/tmp/dati.bin"
12 #define N           100
13
14 static sem_t *s_ready, *s_done;
15
16 static void cleanup(int sig) {
17     (void)sig;
18     if (s_ready) sem_close(s_ready);
19     if (s_done)  sem_close(s_done);
20     /* il writer NON fa unlink: lo fa il reader che li ha creati */
21     _exit(0);
22 }
23
24 int main(void) {
25     signal(SIGINT, cleanup);
26
27     /* TODO: apri i semafori gia' creati dal reader.
28      *      s_ready = sem_open(SEM_READY, 0);
29      *      s_done  = sem_open(SEM_DONE, 0);          */
30
31     printf("[writer] in attesa che il reader sia pronto...\n");
32     /* TODO: sem_wait(s_ready) */
33
34     printf("[writer] scrivo %d interi in %s\n", N, DATAFILE);
35     /* TODO: apri DATAFILE in scrittura (open + O_WRONLY/O_CREAT),
36      *      scrivi N interi casuali con write(), chiudi il file. */
37
38     printf("[writer] scrittura completata, segnalo il reader\n");
39     /* TODO: sem_post(s_done) */
40
41     sem_close(s_ready);
42     sem_close(s_done);
43     return 0;
44 }

```

Listing 2: reader.c – da completare

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <semaphore.h>
5  #include <signal.h>
6  #include <unistd.h>
7
8  #define SEM_READY  "/sem_ready"
9  #define SEM_DONE   "/sem_done"
10 #define DATAFILE  "/tmp/dati.bin"
11 #define N           100

```

```

12
13 static sem_t *s_ready, *s_done;
14
15 static void cleanup(int sig) {
16     (void)sig;
17     if (s_ready) { sem_close(s_ready); sem_unlink(SEM_READY); }
18     if (s_done) { sem_close(s_done); sem_unlink(SEM_DONE); }
19     unlink(DATAFILE);
20     _exit(0);
21 }
22
23 int main(void) {
24     signal(SIGINT, cleanup);
25
26     /* TODO: crea entrambi i semafori.
27      * s_ready = sem_open(SEM_READY, O_CREAT|O_EXCL, 0666, 0);
28      * s_done = sem_open(SEM_DONE, O_CREAT|O_EXCL, 0666, 0); */
29
30     printf("[reader] pronto, segnalo il writer\n");
31     /* TODO: sem_post(s_ready) */
32
33     printf("[reader] attendo che il writer finisca...\n");
34     /* TODO: sem_wait(s_done) */
35
36     printf("[reader] leggo i dati e cerco il massimo\n");
37     /* TODO: apri DATAFILE in lettura, leggi N interi,
38      * trova il massimo e stampalo. */
39
40     /* pulizia finale */
41     sem_close(s_ready); sem_unlink(SEM_READY);
42     sem_close(s_done); sem_unlink(SEM_DONE);
43     unlink(DATAFILE);
44     return 0;
45 }

```

► Ordine di avvio

Il reader deve partire *prima* del writer: è il reader a creare i semafori con `O_CREAT`. Se il writer venisse avviato per primo, la `sem_open` senza `O_CREAT` fallirebbe con `ENOENT`.

[!] Semafori residui in /dev/shm

Se il programma viene interrotto prima della `sem_unlink`, i semafori rimangono visibili in `/dev/shm`. La prossima esecuzione fallirà su `O_EXCL`. Verificare con `ls /dev/shm` e rimuovere manualmente:

```
rm /dev/shm/sem.sem_ready /dev/shm/sem.sem_done
```

In alternativa, rimuovere `O_EXCL` dalla `sem_open` del reader per rendere l'avvio idempotente.

Compilazione e test

```

gcc -Wall -Wextra -o reader reader.c -lrt
gcc -Wall -Wextra -o writer writer.c -lrt

# Terminale 1:
./reader

# Terminale 2 (subito dopo):
./writer

```

Output atteso:

```
# reader
[reader] pronto, segnalo il writer
[reader] attendo che il writer finisca...
[reader] leggo i dati e cerco il massimo
massimo = 998

# writer
[writer] in attesa che il reader sia pronto...
[writer] scrivo 100 interi in /tmp/dati.bin
[writer] scrittura completata, segnalo il reader
```

▷ **Flag di compilazione da ricordare**

- `-lrt`: richiesto da `shm_open`, `sem_open` e tutte le API POSIX real-time.
- Su Linux con `glibc ≥ 2.17` `-lrt` è spesso opzionale, ma è buona pratica linkarlo esplicitamente.